ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΝΑΥΠΗΓΩΝ ΜΗΧΑΝΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ
ΤΟΜΕΑΣ ΜΕΛΕΤΗΣ ΠΛΟΙΟΥ & ΘΑΛΑΣΣΙΩΝ ΜΕΤΑΦΟΡΩΝ

Ακαδημαϊκό Έτος 2025–2026 – Εαρινό Εξάμηνο
«Συστήματα CAD/CAM για την Σχεδίαση & Κατασκευή Πλοίου»

Τεχνική Αναφορά

Αρχιτεκτονική Λογισμικού Παραμετρικής Σχεδίασης Γάστρας σε Rhino/Python

Συγκριτική ανάλυση τριών εκδοχών του κώδικα

`nm23035_cad_cam_ooc_v17_optimized.py`

`nm23035_cad_cam_ooc_v17_copy.py`

`rhino_new_8.py`

Δρίζης Αθανάσιος
Αριθμός Μητρώου: ΝΜ23035

2 Ιουλίου 2026

Περιεχόμενα

1 Εισαγωγή και σκοπός της αναφοράς **2**

2 Περιβάλλον εκτέλεσης και γενική ροή εργασίας **2**

3 Κοινή αρχιτεκτονική και των τριών εκδοχών **3**

3.1 Ιστορικό εξέλιξης μέσα στον ίδιο τον κώδικα 4
3.2 Είσοδος δεδομένων και διαστατοποίηση 5
3.3 Αντικειμενοστραφής πυρήνας: `MyPoint` 7
3.4 `MySegment` και `MyCurve`: από σημεία σε γεωμετρία Rhino 8
3.5 Αλγόριθμος εξομάλυνσης: `universal_fair_function` 9
3.6 Ίσαλοι και ο βρόχος «cross-fairing» 9
3.7 Ομαδοποίηση νομέων και εντοπισμός ασυνέχειας 11
3.8 Παραγωγή επιφανειών: ο κοινός σκελετός της `MySurface` 11

4 Το σημείο απόκλισης **12**

5 `nm23035_cad_cam_ooc_v17_optimized.py` και `..._v17_copy.py`: υλοποιητικές διαφορές **13**

5.1 Δημιουργία σημείων Rhino: *lazy* με caching έναντι *eager* χωρίς caching 13
5.2 Πρόβλημα κωδικοποίησης χαρακτήρων στο **`v17_copy.py`** 14
5.3 Κοινή στρατηγική πρύμνης 15

6 `rhino_new_8.py`: γενικευμένη αρχιτεκτονική **15**

6.1 Πύκνωση πλέγματος νομέων: `create_intermediate_stations` 15
6.2 Γενίκευση της `MySurface` για απευθείας χειρισμό ασυνεχών νομέων . . 16
6.3 Ενοποιημένη στρατηγική πρύμνης: *blended loft* αντί για δίκτυο επιφανειών 17

7 Συγκεντρωτική σύγκριση **18**

8 Συμπεράσματα **19**

1 Εισαγωγή και σκοπός της αναφοράς

Η παρούσα αναφορά τεκμηριώνει την αρχιτεκτονική του λογισμικού που αναπτύχθηκε στο πλαίσιο της εργαστηριακής άσκησης του μαθήματος «Συστήματα CAD/CAM για την Σχεδίαση & Κατασκευή Πλοίου». Αντικείμενο της άσκησης είναι η αυτόματη παραγωγή τρισδιάστατου μοντέλου CAD της γάστρας ενός πατρικού αδιάστατου πλοίου τύπου Bulk Carrier, με δεδομένα σημεία νομέων, προφίλ, επίπεδου τμήματος πυθμένα/πλευράς (FOB/FOS) και καθρέφτη πρύμνης, διαστατοποιημένα σύμφωνα με τα προσωπικά στοιχεία του φοιτητή (πλήθος γραμμάτων ονόματος, τελευταίο ψηφίο αριθμού μητρώου).

Η υλοποίηση έγινε σε Python, με συγγραφή του κώδικα σε VS Code και εκτέλεσή του μέσα από το Script Editor του Rhinoceros 8 (παρακάμπτοντας το γραφικό περιβάλλον του Grasshopper), κάνοντας χρήση των βιβλιοθηκών `rhinoscriptsyntax` και `Rhino.Geometry`. Μου δόθηκαν προς ανάλυση τρία αρχεία κώδικα που αντιστοιχούν σε διαφορετικά στιγμιότυπα/κλάδους της ίδιας προσπάθειας:

- **`nm23035_cad_cam_ooc_v17_optimized.py`** – 2712 γραμμές,
- **`nm23035_cad_cam_ooc_v17_copy.py`** – 2655 γραμμές,
- **`rhino_new_8.py`** – 3035 γραμμές.

Και τα τρία αρχεία μοιράζονται τον ίδιο πυρήνα (ίδιες συναρτήσεις, ίδιες κλάσεις, ίδια σειρά επεξεργαστικών σταδίων), κάτι που επιβεβαιώνεται τόσο από το ταυτόσημο ιστορικό σχολίων εκδόσεων στην κορυφή κάθε αρχείου (βλ. §3.1) όσο και από τη σχεδόν γραμμή-προς-γραμμή αντιστοιχία των ενοτήτων τους. Η δομή της αναφοράς ακολουθεί αυτήν ακριβώς την παρατήρηση: στο Κεφάλαιο 3 περιγράφεται η αρχιτεκτονική που είναι κοινή και στα τρία αρχεία. Στο Κεφάλαιο 4 εντοπίζεται το σημείο και ο τρόπος της απόκλισης, και στα Κεφάλαια 5–6 εξετάζεται ξεχωριστά η κάθε εκδοχή, εστιάζοντας αποκλειστικά σε ό,τι τη διαφοροποιεί από τις υπόλοιπες.

Παρατήρηση εμβέλειας: τα Ζητούμενα 1–4 της εκφώνησης (διαστατοποίηση, παρεμβολή/εξομάλυνση βασικών καμπυλών, παρεμβολή/εξομάλυνση/διόρθωση νομέων, κατασκευή επιφανειακών τμημάτων με συνέχεια G^1) καλύπτονται πλήρως και από τα τρία αρχεία. Το Ζητούμενο 5 (υδροστατικές καμπύλες: εκτόπισμα, LCB, TCB, WS, A_w, LCF) δεν εμφανίζεται ως υπολογιστικός κώδικας σε κανένα από τα τρία αρχεία – προφανώς επειδή υπολογίζεται απευθείας μέσα από τα εργαλεία όγκου/εμβαδού/κέντρου βάρους του Rhino πάνω στο τελικό, κατοπτρισμένο μοντέλο, και δεν αποτελεί μέρος του αυτοματοποιημένου pipeline γεωμετρίας.

2 Περιβάλλον εκτέλεσης και γενική ροή εργασίας

Και οι τρεις εκδοχές ξεκινούν με το ίδιο εισαγωγικό *docstring*, το οποίο περιγράφει τον τρόπο εργασίας:

```
"""nm23035_cad_cam_v1εργασία
 συστημάτων cad/cam Drizis Athanasios nm23035η
 εργασία γράφεται στο vs code και στη συνέχεια κάνει run στο script editorτου
 rhino 8, κατ αυτό το τρόπο κάνοντας bypass το περιβάλλον του grasshopperσκοπός
 του παρακάτω script είναι η αυτόματη σχεδίαση της γεωμετρίας τηςγάστρας
 και ο υπολογισμός των απαιτούμενων μεγεθών βλέπε( εκφώνηση), τα
inputs θα είναι το πλήθος γραμμάτων του ονόματος σας και το τελευταίοψηφίο
 του αριθμού μητρώουκαλή
 μου τύχη"""
```

Listing 1: Κεφαλίδα τεκμηρίωσης (ταυτόσημη και στα τρία αρχεία)

Το script δεν είναι αυτόνομο εκτελέσιμο πρόγραμμα με γραφικό interface, αλλά ένα διαδοχικό (top-to-bottom) script διαδικαστικού τύπου, στο οποίο σχεδόν όλες οι μεταβλητές είναι *module-level* και εκτελούνται με τη σειρά που εμφανίζονται στο αρχείο. Ο μόνος διάλογος με τον χρήστη γίνεται στην αρχή, μέσω `rs.GetInteger()` (καθώς το `input()` της Python «παγώνει» το περιβάλλον του Rhino), και αφορά τα δύο μεγέθη διαστατοποίησης:

```
number_of_letters = rs.GetInteger("number of letters in first name")
last_number_of_nm = rs.GetInteger("last number of your nm")

L_BP0 = 225
B_HALF0 = 15.9
l_bp = L_BP0 + number_of_letters
b_half = B_HALF0 + (last_number_of_nm/10.0)
d = b_half * 0.62
b_whole = 2 * b_half
```

Listing 2: Είσοδος χρήστη και υπολογισμός κύριων διαστάσεων

Ενδεικτικά, για το όνομα «Αθανάσιος» ($O = 9$ γράμματα) και ΑΜ NM23035 ($A = 5$), προκύπτουν $L_{BP} = 234\,\text{m}$, $B/2 = 16.4\,\text{m}$, $D \approx 10.17\,\text{m}$ – οι πραγματικές τιμές όμως καθορίζονται κάθε φορά διαδραστικά κατά την εκτέλεση και δεν είναι «σκληρά κωδικοποιημένες» (hard-coded) πουθενά στο script.

Πριν από οτιδήποτε άλλο, γίνεται καθαρισμός του εγγράφου Rhino (`rs.AllObjects()` + `rs.DeleteObjects()`), ώστε κάθε εκτέλεση να ξεκινά από κενό canvas – απαραίτητο αφού το script τρέχει επανειλημμένα κατά τη διάρκεια της ανάπτυξης.

3 Κοινή αρχιτεκτονική και των τριών εκδοχών

Η γενική ροή επεξεργασίας – από τα ακατέργαστα αδιάστατα σημεία μέχρι το τελικό, κατοπτρισμένο μοντέλο γάστρας – είναι πανομοιότυπη και στα τρία αρχεία και οργανώνεται σε εννέα διαδοχικά στάδια:

1. Φόρτωση αδιάστατων σημείων (offsets, profile, FOS, FOB, transom) ως πολυγραμμικά *string literals*.
2. Διαστατοποίηση και ταξινόμηση σημείων ανά τύπο καμπύλης.
3. Κατασκευή του αντικειμενοστραφούς μοντέλου σημείων/τμημάτων/καμπυλών (`MyPoint`, `MySegment`, `MyCurve`).

4. Εξομάλυνση (*fairing*) βασικών καμπυλών ορίου (profile, FOS, FOB, transom).
5. Κατασκευή αρχικών νομέων από τις τομές με τις καμπύλες ορίου.
6. Επαναληπτικός βρόχος «cross-fairing» ισάλων/νομέων.
7. Ομαδοποίηση νομέων και εντοπισμός ασυνεχειών (πολύπλοκη πρύμνη).
8. Παραγωγή επιφανειακών τμημάτων (`MySurface`) και κατοπτρισμός.
9. Ειδική διαχείριση καθρέφτη πρύμνης (transom) και ασυνεχών νομέων πρύμνης.

Στη συνέχεια αναλύεται καθένα από αυτά τα στάδια όπως εμφανίζεται –ταυτόσημο ή σχεδόν ταυτόσημο– και στα τρία αρχεία.

3.1 Ιστορικό εξέλιξης μέσα στον ίδιο τον κώδικα

Ένα ιδιαίτερο χαρακτηριστικό και των τριών αρχείων είναι ότι διατηρούν, ως σχόλια στην κορυφή τους, ένα αναλυτικό *changelog* των εκδόσεων V2 έως V14 που προηγήθηκαν της τρέχουσας μορφής του κώδικα. Το changelog είναι ταυτόσημο και στα τρία αρχεία, γεγονός που επιβεβαιώνει ότι πρόκειται για κλάδους (*branches*) της ίδιας ιστορικής γραμμής ανάπτυξης και όχι για ανεξάρτητες υλοποιήσεις:

Έκδοση	Περιεχόμενο σχολίου (περίληψη)
V2	Εντοπισμός επανάληψης κώδικα στη δημιουργία λιστών σημείων (fob/fos/profile/offsets)· σχεδιασμός μίας γενικής συνάρτησης με παράμετρο δείκτη ταξινόμησης.
V3	Διόρθωση προβλήματος ταξινόμησης σημείων του profile (λανθασμένη γραμμή λόγω ταξινόμησης με αύξουσα τιμή x)· εισαγωγή ταξινόμησης πρύμνη-πλώρη σε δύο τμήματα.
V5	Παρατήρηση ότι η αύξηση του αριθμού δειγμάτων μεγαλώνει τεχνητά την ενέργεια κάμψης (*bending energy*)· έναρξη διόρθωσης.
V6	Ολοκλήρωση όλων των καμπυλών νομέων εντός των ορίων γάστρας, με προσθήκη «φαντασματικών» σημείων (ghost points) στις FOS/FOB/Profile για G^2 συνέχεια· διάσπαση νομέων σε τμήματα πρύμνης/πλώρης λόγω πολύπλοκης γεωμετρίας.
V7	Έναρξη διαδικασίας εξομάλυνσης (fairing).
V8	Πλήρης επανασχεδίαση σε αντικειμενοστραφή μορφή (Object-Oriented).
V9	Συνέχιση της αντικειμενοστραφούς επανασχεδίασης.
V12	Διόρθωση συνθήκης εφαπτομένης στα όρια καμπύλης (τα V10–V11 χαρακτηρίζονται ως αποτυχημένα)· διόρθωση νομέων με τελικό σημείο εκτός καμπύλης, ώστε να μην υπάρχουν «τρύπες» στη γεωμετρία, αγνοώντας πολύ κοντινά αρχικά σημεία σε σχέση με σημεία τομής.
V13	Ολοκλήρωση λογικής ισάλων (waterlines).
V14	Ολοκλήρωση λογικής «cross-fairing».

Η ονομασία **`..._v17_optimized`** / **`..._v17_copy`** υποδηλώνει ότι οι δύο αυτές εκδοχές προέρχονται από ένα κοινό σημείο V17, ενώ το **`rhino_new_8.py`** – παρότι το όνομά του παραπέμπει σε παλαιότερο αριθμό έκδοσης – περιέχει, όπως θα φανεί στο Κεφάλαιο 6, πρόσθετη λειτουργικότητα που δεν υπάρχει στα άλλα δύο αρχεία. Η αναφορά δεν υποθέτει αυστηρή χρονική σειρά μεταξύ των τριών αρχείων· τα παρουσιάζει ως τρεις παράλληλες παραλλαγές του ίδιου πυρήνα και τεκμηριώνει τις πραγματικές τεχνικές διαφορές τους.

3.2 Είσοδος δεδομένων και διαστατοποίηση

Τα ακατέργαστα σημεία κάθε καμπύλης (offsets/νομείς, profile, FOB, FOS, transom) είναι ενσωματωμένα στον κώδικα ως πολυγραμμικά strings της μορφής `"x,y,z"` ανά γραμμή, σε αδιάστατη μορφή (τιμές στο $[0, 1]$ περίπου). Τρεις συναρτήσεις υλοποιούν, με τον ίδιο τρόπο και στα τρία αρχεία, το πέρασμα από αδιάστατα δεδομένα σε αντικείμενα σημείων:

- `get_limits(profile_data_string, L_scale, B_half, D_scale)` – υπολογίζει τον πίνακα ορίων γάστρας 2×3 (ελάχιστο/μέγιστο x, y, z σε μέτρα), χρησιμοποιώντας το ελάχιστο x του profile για το όριο της πρύμνης.
- `get_points_from_data(...)` – η «καθολική» συνάρτηση διαστατοποίησης/ταξινόμησης που προέκυψε από την παρατήρηση της V2: παίρνει ως όρισμα τον τύπο καμπύλης (`"fob"`, `"fos"`, `"profile"`, `"transom"`, `"station"`) και εφαρμόζει διαφορετική λογική ταξινόμησης ανά περίπτωση.
- `get_point_objects(point_data, station_flag)` – μετατρέπει τις πλειάδες δεδομένων σε αντικείμενα `MyPoint` και τα συνδέει σε διπλά συνδεδεμένη λίστα μέσω `.prev` / `.next`.

Η ταξινόμηση των σημείων του *profile* αξίζει ιδιαίτερη αναφορά, καθώς αποτέλεσε αντικείμενο της διόρθωσης V3: το profile χωρίζεται σε πρυμναίο και πλωραίο μισό ως προς $x = L/2$, με το πρυμναίο μισό ταξινομημένο κατά *φθίνουσα* τιμή z και το πλωραίο κατά *αύξουσα* τιμή z, ώστε τα διαδοχικά σημεία να σχηματίζουν συνεχή γραμμή γύρω από το περίγραμμα του πλοίου αντί για μια «θορυβώδη» ζιγκζαγκ γραμμή:

```
if curve_type == "profile":
    aft_part = [p for p in all_points if p[0] < L_scale/2]
    fwd_part = [p for p in all_points if p[0] >= L_scale/2]
    aft_part.sort(key=lambda p: p[2], reverse=True)
    fwd_part.sort(key=lambda p: p[2])
    all_points = aft_part + fwd_part
```

Listing 3: Ταξινόμηση σημείων profile (κοινή λογική και στα τρία αρχεία)

Η περίπτωση `curve_type == "station"` είναι η πιο σύνθετη: τα σημεία των νομέων δίνονται όλα μαζί σε ένα ενιαίο σύνολο (`offsets_nodim`), χωρίς να δηλώνεται εκ των προτέρων ποιο σημείο ανήκει σε ποιον νομέα. Ο διαχωρισμός γίνεται με ταξινόμηση κατά x και ομαδοποίηση διαδοχικών σημείων με απόσταση $|x_i - x_{i-1}| < 0.01$ (ανοχή λόγω σφαλμάτων κινητής υποδιαστολής από τους πολλαπλασιασμούς διαστατοποίησης), όπως περιγράφεται και στο ίδιο το σχόλιο του κώδικα:

```
all_points.sort(key=lambda p: p[0])
current_station = [all_points[0]]
for i in range(1, len(all_points)):
    # σε περίπτωση που με τους πολλαπλασιασμούς έχει ξεφύγει κάποιο
    # νούμερο δεν χρησιμοποιώ ισότητα, δέχομαι διαφορά στο x έως 1 εκ.
    if abs(all_points[i][0] - all_points[i-1][0]) < 0.01:
        current_station.append(all_points[i])
    else:
        current_station.sort(key=lambda p: p[2])
        stations.append(current_station)
        current_station = [all_points[i]]
current_station.sort(key=lambda p: p[2])
stations.append(current_station)
```

Listing 4: Ομαδοποίηση σημείων σε νομείς βάσει x

3.3 Αντικειμενοστραφής πυρήνας: `MyPoint`

Η κλάση `MyPoint` είναι ο θεμέλιος λίθος και των τριών εκδοχών· κάθε αρχικό ή προερχόμενο από τομή (*intercept*) σημείο της γεωμετρίας αναπαρίσταται ως στιγμιότυπο αυτής της κλάσης, η οποία «γνωρίζει» πού βρίσκεται στον χώρο, ποιους περιορισμούς ικανοποιεί, και ποια εφαπτομενική συνθήκη πρέπει να επιβληθεί εκεί ώστε η τελική επιφάνεια να έχει συνέχεια G^1. Πρόκειται ουσιαστικά για ένα πλούσιο σε ιδιότητες (*properties*) «έξυπνο σημείο» αντί για απλή τριάδα συντεταγμένων, με δομή *doubly linked list* μέσω των πεδίων `.prev`/`.next`, ώστε κάθε σημείο να μπορεί να «βλέπει» τη γειτονιά του.

Οι σημαντικότερες ιδιότητες (properties) της κλάσης, κοινές και στις τρεις εκδοχές, είναι:

- `coords` – επιστρέφει/ορίζει τη θέση του σημείου (`rg.Point3d`).
- `limits_bool_list` – πίνακας 3×2 (άξονας $\times$ min/max) που δηλώνει αν το σημείο βρίσκεται πάνω σε κάποιο όριο της γάστρας (transom, centerline, FOS, baseline, deck), με ανοχή 0.001.
- `boundary_hit` – συνθέτει μία ενιαία ετικέτα (π.χ. `"CL+FOS"`) συνδυάζοντας τυχόν χειροκίνητα ορισμένη ετικέτα (`self.hit`) με τα spatial hits του `limits_bool_list`.
- `is_constrained` – επιστρέφει 0 (ελεύθερο), 1 (ένας βαθμός ελευθερίας, «ολισθαίνει» πάνω σε ευθεία) ή 2 (πλήρως δεσμευμένο), ανάλογα με το πλήθος των ενεργών περιορισμών.
- `is_line` – πίνακας [πριν, μετά] που δηλώνει αν η σύνδεση με τον προηγούμενο-/επόμενο γείτονα είναι ευθύγραμμη (σταθερή συντεταγμένη κάθετη στο επίπεδο της καμπύλης).
- `is_tangent` / `line_directions` / `is_knuckle` – καθορίζουν αν και προς ποια κατεύθυνση απαιτείται δεσμευμένη εφαπτομένη, με ειδική εξαίρεση για τα σημεία «άρθρωσης» (*knuckle points*) της πρύμνης, όπου η σύνδεση είναι κατακόρυφη ($|d_x| < 0.01,\ |d_z| > 0.99$) και η καμπύλη *πρέπει* να αλλάζει κατεύθυνση απότομα.
- `required_tangent_vector` – συγκεντρώνει όλη την παραπάνω πληροφορία σε ένα ενιαίο διάνυσμα εφαπτομένης (ή `None`), με διαφορετική λογική ανά τύπο σημείου (`station`, `transom`, `waterline`, `profile`).

```
if pt_type_lower == "station":
    if "CL" in hit or "BL" in hit:
        ref = self.next if self.next else self.prev
        if ref:
            raw_dir = (self.next.coords - self.coords) if self.next \
                      else (self.coords - self.prev.coords)
            projected = rg.Vector3d(raw_dir.X, raw_dir.Y, 0.0)
            if projected.Length > 0.0001:
                projected.Unitize()
                return projected
        return rg.Vector3d(0, 1, 0)
    if "FOS" in hit:
```

```
        ...
        projected = rg.Vector3d(raw_dir.X, 0.0, raw_dir.Z)
        ...
```

Listing 5: MyPoint: υπολογισμός δεσμευμένης εφαπτομένης σε σημεία νομέα (απόσπασμα)

Η λογική αυτή είναι που επιτρέπει στο σύστημα να «γνωρίζει» μόνο του, χωρίς χειροκίνητη παρέμβαση ανά νομέα, πότε ένα σημείο πρέπει να έχει οριζόντια εφαπτομένη (σημείο στο επίπεδο τμήμα πυθμένα/πλευράς) και πότε κατακόρυφη ή ελεύθερη – κρίσιμη προϋπόθεση για τη συνέχεια G^1 που ζητά η εκφώνηση (Ζητούμενο 4).

3.4 `MySegment` και `MyCurve`: από σημεία σε γεωμετρία Rhino

Η κλάση `MySegment` «περιτυλίγει» ένα πραγματικό αντικείμενο γεωμετρίας Rhino (GUID), συνδέοντάς το με τα δύο άκρα του σε μορφή `MyPoint`, ώστε να είναι άμεσα διαθέσιμες ιδιότητες όπως `start_tangent`, `end_hit`, `start_is_knuckle` χωρίς επανάληψη κώδικα κάθε φορά που χρειάζεται αυτή η πληροφορία. Η μέθοδος `boundary_hits` επιστρέφει την ένωση των ετικετών ορίου των δύο άκρων, χρησιμοποιούμενη εκτενώς στη συνέχεια για την ταξινόμηση τμημάτων ανά συνοριακή συνθήκη (π.χ. «ποια τμήματα ακουμπούν το FOB;»).

Η κλάση `MyCurve` είναι το επίπεδο πάνω από το `MySegment`: αναπαριστά μία πλήρη καμπύλη ορίου ή νομέα ως ακολουθία σημείων `MyPoint`, την οποία μετατρέπει σε πραγματική γεωμετρία Rhino στη μέθοδο `rebuild()`. Η βασική λογική του `rebuild()` είναι: διάσχιση της λίστας σημείων και ομαδοποίηση διαδοχικών σημείων σε «γραμμές» (`LINE`) ή «καμπύλες» (`BSPLINE`) ανάλογα με τη σημαία `is_line`, με κάθε ομάδα να παράγει ένα ξεχωριστό αντικείμενο Rhino και το αντίστοιχο `MySegment`. Πριν από αυτό, η μέθοδος `filter_intercept_overlaps()` αφαιρεί αρχικά σημεία νομέα που βρίσκονται πολύ κοντά (< 0.5 μονάδες) σε σημεία τομής (intercepts) με τις καμπύλες ορίου – η «λύση έκτακτης ανάγκης» («very trashy fix» κατά τα λόγια του ίδιου του σχολίου της V12) που αποτρέπει τρύπες στη γεωμετρία λόγω διπλών/σχεδόν ταυτόσημων σημείων.

Άλλες σημαντικές μέθοδοι της `MyCurve`, κοινές και στα τρία αρχεία:

- `get_intercepts_with_plane(station_x)` – τομή όλων των τμημάτων της καμπύλης με κατακόρυφο επίπεδο YZ σε συγκεκριμένο x (χρησιμοποιείται π.χ. για την εύρεση σημείων profile κατά την επέκταση ελλιπών νομέων).
- `calculate_segment_inflections(segment_id)` – εντοπίζει σημεία καμπυλότητας μηδέν (σημεία καμπής) δειγματοληπτώντας την καμπύλη σε 50 βήματα και παρακολουθώντας το πρόσημο του $\vec{t} \times \vec{\kappa}$.
- `display_report()` / `display_segments_report()` – εργαλεία εντοπισμού σφαλμάτων (*debugging*) που τυπώνουν πλήρη πίνακα κατάστασης ανά σημείο/τμήμα (τύπος, περιορισμοί, εφαπτομένη, συντεταγμένες).

3.5 Αλγόριθμος εξομάλυνσης: `universal_fair_function`

Η εξομάλυνση των καμπυλών (Ζητούμενα 2–3 της εκφώνησης) γίνεται μέσω μίας ενιαίας συνάρτησης που εφαρμόζεται σε κάθε `MySegment` ξεχωριστά και είναι πανομοιότυπη και στα τρία αρχεία. Η λογική της είναι η εξής:

1. Τα ευθύγραμμα τμήματα (`seg.type=="LINE"`) παραλείπονται – δεν έχει νόημα εξομάλυνση σε ευθεία.
2. Το τμήμα ταξινομείται σε `SMALL` (< 10 μονάδες μήκους), `MEDIUM` (10–20) ή `BIG` (> 20), με αντίστοιχο πλήθος σημείων ελέγχου 5, 8 και 12 μετά από `Rebuild(pointCount, degree=3, preserveEndpoints=True)`.
3. Η δέσμευση (*clamping*) κάθε άκρου ρυθμίζεται δυναμικά: αν το άκρο έχει `required_tangent_vector` διάφορο του `None`, ζητείται εφαπτομενική συνέχεια (G^1, clamp$=1$)· διαφορετικά μόνο θέση (G^0, clamp$=0$).
4. Καλείται η μέθοδος `Fair` του `NurbsCurve` του Rhino 8 (υπερφόρτωση 5 παραμέτρων: απόσταση, γωνία $80°$, clamp start/end, 100 επαναλήψεις), και το αποτέλεσμα αντικαθιστά το αρχικό αντικείμενο στο έγγραφο Rhino.

```
clamp_start = 1 if seg.start_tangent is not None else 0
clamp_end   = 1 if seg.end_tangent   is not None else 0

length = nurbs.GetLength()
if length < 10:
    num_of_cp = 5      # SMALL
elif length < 20:
    num_of_cp = 8      # MEDIUM
else:
    num_of_cp = 12     # BIG

nurbs = nurbs.Rebuild(num_of_cp, 3, True)
fair_func = nurbs.Fair.Overloads[System.Double, System.Double,
                                 System.Int32, System.Int32, System.Int32]
success = fair_func(dist, ang, clamp_start, clamp_end, loop)
```

Listing 6: `universal_fair_function`: δυναμική δέσμευση και ταξινόμηση μεγέθους

Η ίδια συνάρτηση χρησιμοποιείται τόσο για τις βασικές καμπύλες ορίου (profile, FOS, FOB) όσο και – μέσα στον επαναληπτικό βρόχο – για τους νομείς και τις ισάλους, γεγονός που αντικατοπτρίζεται και στο όνομά της (*universal*).

3.6 Ίσαλοι και ο βρόχος «cross-fairing»

Η πυρηνική ιδέα πίσω από τη διόρθωση των νομέων (Ζητούμενο 3: «*παρεμβολή, εξομάλυνση, έλεγχος συμβατότητας και διόρθωση των νομέων εάν κριθεί απαραίτητο*») είναι η αμοιβαία, επαναληπτική εξομάλυνση νομέων και ισάλων μέχρι σύγκλισης, μία τεχνική γνωστή στη ναυπηγική σχεδίαση ως *cross-fairing* και η οποία, όπως σημειώνεται στο ιστορικό εκδόσεων, ολοκληρώθηκε στις εκδόσεις V13–V14. Η συνάρτηση `build_waterline_segments(...)` δημιουργεί οριζόντιες καμπύλες ισάλου σε ισοκατανεμημένα ύψη z (ξεκινώντας λίγο πάνω από τη γραμμή βάσης ώστε να

αποφεύγεται το FOB), τέμνοντάς τες με τις καμπύλες FOS, profile, transom και όλους τους τρέχοντες νομείς.

Ο κύριος βρόχος (`total_cycles = 5` επαναλήψεις) εκτελεί εναλλάξ:

```
for cycle in range(total_cycles):
    # --- ΒΗΜΑ Α: ΙΣΑΛΟΙ παρεμβολή( -> εξομάλυνση) ---
    purge_geometry_by_name(current_waterlines)
    current_waterlines = build_waterline_segments(
        my_fos_curve_object, my_profile_curve_object,
        my_transom_curve_object, current_stations,
        ship_limits_matrix, num_waterlines)
    for wl in current_waterlines:
        for seg in wl.segments:
            universal_fair_function(seg)
    clamp_waterlines_y(current_waterlines)

    # --- ΒΗΜΑ Β: ΝΟΜΕΙΣ ανακατασκευή( -> επανεξομάλυνση) ---
    new_stations = rebuild_stations_with_waterlines(
        current_stations, current_waterlines, ship_limits_matrix)
    purge_geometry_by_name(current_stations)
    current_stations = new_stations
    for st in current_stations:
        for seg in st.segments:
            universal_fair_function(seg)
```

Listing 7: Ο βρόχος cross-fairing (κοινός και στα τρία αρχεία)

Δηλαδή: οι νομείς παράγουν τις ισάλους → οι ίσαλοι εξομαλύνονται → οι εξομαλυμένοι ίσαλοι ξαναχτίζουν τους νομείς → οι νέοι νομείς εξομαλύνονται → η διαδικασία επαναλαμβάνεται. Επειδή η εξομάλυνση μετατοπίζει ελαφρώς τα σημεία, μετά το τέλος του βρόχου οι δύο οικογένειες καμπυλών (νομείς/ίσαλοι) δεν τέμνονται πλέον με απόλυτη ακρίβεια. Γι' αυτό ακολουθεί ένα τελικό, **μη-εξομαλυμένο** πέρασμα («Final Interpolation Pass»): οι ίσαλοι ξαναχτίζονται μία τελευταία φορά μέσα από τους ήδη εξομαλυμένους νομείς, χωρίς να περάσουν ξανά από `universal_fair_function`, ώστε να «κλειδώσουν» ακριβώς πάνω στους τελικούς νομείς και να εξασφαλιστεί σωστή τομή για τη μετέπειτα δημιουργία επιφανειών:

```
# Rebuild the waterlines ONE last time -- interpolated through the
# smoothed stations, but NOT faired -- so every station/waterline
# pair is guaranteed to intersect for surfacing.
current_waterlines = build_waterline_segments(
    my_fos_curve_object, my_profile_curve_object,
    my_transom_curve_object, current_stations,
    ship_limits_matrix, num_waterlines)
clamp_waterlines_y(current_waterlines)
```

Listing 8: Τελικό πέρασμα «κλειδώματος» ισάλων χωρίς εξομάλυνση

Η βοηθητική συνάρτηση `purge_geometry_by_name(curve_list)` είναι απαραίτητη επειδή η εντολή `Fair` του Rhino αλλάζει το εσωτερικό GUID ενός αντικειμένου διατηρώντας όμως το όνομά του· επομένως ο καθαρισμός γεωμετρίας πριν από κάθε νέο κύκλο δεν μπορεί να βασιστεί στα αποθηκευμένα ID, παρά μόνο σε αναζήτηση βάσει ονόματος (`rs.ObjectsByName`).

3.7 Ομαδοποίηση νομέων και εντοπισμός ασυνέχειας

Στην πολύπλοκη γεωμετρία της πρύμνης, ένας νομέας μπορεί να «σπάσει» σε περισσότερα του ενός ασύνδετα τμήματα (*parts*) – ήδη από τη V6 προβλέφθηκε η «διάσπαση νομέων σε τμήματα πρύμνης/πλώρης λόγω πολύπλοκης γεωμετρίας». Τρεις συναρτήσεις, κοινές και στα τρία αρχεία, διαχειρίζονται αυτή την περίπτωση πριν την παραγωγή επιφανειών:

- `group_stations_by_number(station_list)` – ομαδοποιεί όλα τα αντικείμενα `MyCurve` που ανήκουν στον ίδιο αριθμό νομέα (εξάγοντας τον αριθμό από το όνομα, π.χ. `"Station_7_Part_1"`).
- `select_representative_station(station_group)` – επιλέγει, από κάθε ομάδα, το τμήμα με τα περισσότερα segments ως «αντιπροσωπευτικό» νομέα για τη γενική ροή lofting.
- `check_station_discontinuity(station_group)` – επιστρέφει `True` αν εντοπίζονται περισσότερα από ένα `Part_N` στο όνομα των καμπυλών της ομάδας, δηλαδή αν ο νομέας είναι ασυνεχής.

Το αποτέλεσμα αυτού του σταδίου είναι μία λίστα `processed_stations` από λεξικά της μορφής `{'number': ..., 'curve': ..., 'discontinuous': ...}`, η οποία τροφοδοτεί απευθείας το επόμενο στάδιο παραγωγής επιφανειών.

3.8 Παραγωγή επιφανειών: ο κοινός σκελετός της `MySurface`

Η κλάση `MySurface` αναλαμβάνει την κατασκευή ενός επιφανειακού τμήματος ανάμεσα σε δύο διαδοχικούς νομείς (Ζητούμενο 4). Ο κατασκευαστής της είναι πανομοιότυπος και στα τρία αρχεία και αποθηκεύει αναφορές στους δύο νομείς-άκρα καθώς και στις τέσσερις καμπύλες ορίου (profile, FOB, FOS, transom), απαραίτητες για την επέκταση ελλιπών νομέων. Δύο μέθοδοι είναι κοινές, τόσο σε υπογραφή όσο και σε υλοποίηση, και στα τρία αρχεία:

- `_check_waterproof(station_obj)` – ελέγχει αν ο νομέας «κλείνει στεγανά» στην γραμμή βάσης/κεντρική γραμμή ($Y \approx 0$ στο πρώτο σημείο) και στο κατάστρωμα ($Z \approx Z_{max}$ στο τελευταίο σημείο), επιστρέφοντας `[bool, bool]`.
- `_extend_station(station_obj)` – αν ο νομέας δεν είναι στεγανός, προσθέτει τεχνητά ευθύγραμμα τμήματα επέκτασης προς τα κάτω (μέχρι το profile, μέσω `get_intercepts_with_plane`) ή προς τα πάνω (μέχρι το κατάστρωμα, $Z = Z_{max}$), κατασκευάζοντας τα αντίστοιχα προσωρινά `MyPoint`/`MySegment`.

Το επιφανειακό τμήμα κατασκευάζεται με `rg.Brep.CreateFromLoft` ανάμεσα στις δύο (ενωμένες μέσω `rs.JoinCurves`) καμπύλες νομέα, με το επιφανειακό τμήμα της αριστερής πλευράς (port) να παράγεται πρώτο και στη συνέχεια να κατοπτρίζεται ως προς το επίπεδο ZX (`rg.Transform.Mirror(rg.Plane.WorldZX)`) για τη δημιουργία της δεξιάς πλευράς (starboard) – υλοποιώντας έτσι το Ζητούμενο 5 του κατοπτρισμού.

Στη γενική ροή («MAIN SURFACE GENERATION»), οι διαδοχικές, *συνεχείς* στήλες νομέων συνδέονται η μία μετά την άλλη με στιγμιότυπα `MySurface`, ενώ ασυνεχείς νομείς προκαλούν παράλειψη («SKIPPED: Discontinuous») στη γενική αυτή ροή – η ειδική τους διαχείριση γίνεται ξεχωριστά, όπως περιγράφεται στο §4. Στην πλώρη, μετά τον τελευταίο συνεχή νομέα, χρησιμοποιείται αντ' αυτού `rs.AddNetworkSrf` με U-καμπύλες τον τελευταίο νομέα και το (κομμένο στο σημείο εκείνο) profile, και V-καμπύλες όσους ισάλους τέμνουν το πλωραίο τμήμα του profile – προσέγγιση κοινή και στα τρία αρχεία (ενότητα «FORWARD BOW SURFACE»).

4 Το σημείο απόκλισης

Η σύγκριση σε επίπεδο γραμμών (`diff`) ανάμεσα στα δύο αρχεία **`v17_optimized`** και **`v17_copy`** δείχνει ότι **το σύνολο των ονομάτων συναρτήσεων, κλάσεων και μεθόδων παραμένει ταυτόσημο**, με τις ενότητες κώδικα να εμφανίζονται στην ίδια ακριβώς σειρά (με απόκλιση λίγων μόνο γραμμών λόγω κενών/σχολίων). Η απόκλιση των δύο αυτών αρχείων είναι επομένως **υλοποιητική, όχι αρχιτεκτονική**: αφορά *πώς* υλοποιείται η ίδια λογική (π.χ. lazy vs. eager δημιουργία σημείων Rhino, ύπαρξη ή μη caching) και όχι *τι* κάνει το πρόγραμμα. Αναλύονται στο Κεφάλαιο 5 (κοινό με το 5).

Το **`rhino_new_8.py`** αποκλίνει πιο ουσιαστικά, και το κάνει σταδιακά:

- Μέχρι και το στάδιο του cross-fairing βρόχου (§3.6), η αρχιτεκτονική παραμένει ουσιαστικά η ίδια.
- Αμέσως μετά, εμφανίζεται μία εντελώς νέα συνάρτηση, `create_intermediate_stations(...)`, που δεν υπάρχει στα άλλα δύο αρχεία (βλ. §6.1).
- Η κλάση `MySurface` διατηρεί μεν τον κοινό σκελετό της §3.8, αλλά αποκτά πέντε επιπλέον μεθόδους (`_collect_segs`, `_collect_segs_by_part`, `_trim_curve_by_z`, `_loft_pair`, `_close_gaps`) που της επιτρέπουν να χειρίζεται *απευθείας* ασυνεχείς νομείς, αντί να τους παραλείπει (§6.2).
- Η τελική ενότητα διαχείρισης της πρύμνης αντικαθίσταται πλήρως: αντί για διαδοχικές επιφάνειες δικτύου (*network surfaces*) ανά τμήμα νομέα, χρησιμοποιείται ένα ενιαίο *blended loft* από τον καθρέφτη πρύμνης έως τον πρώτο νομέα (§6.3).

Ο παρακάτω πίνακας συνοψίζει το σημείο και τη φύση της απόκλισης:

	v17_optimized / v17_copy	rhino_new_8
Σύνολο συναρτήσεων/κλάσεων	Ταυτόσημο	Υπερσύνολο (επιπλέον 6 συναρτήσεις/μέθοδοι)
Δημιουργία `MyPoint`	Διαφέρει (lazy/eager)	Όπως `v17_optimized`
Ασυνεχείς νομείς πρύμνης	Παραλείπονται στη γενική ροή, χειρισμός σε ξεχωριστό βήμα	Ενσωματωμένοι στη γενική ροή `MySurface`
Πύκνωση πλέγματος νομέων	Δεν υπάρχει	`create_intermediate_stations`
Τελική επιφάνεια πρύμνης	Δίκτυο επιφανειών ανά τμήμα (`AddNetworkSrf`)	Ενιαίο blended loft (`CreateFromLoft`)

5 `nm23035_cad_cam_ooc_v17_optimized.py` και `..._v17_copy.py`: υλοποιητικές διαφορές

Καθώς η αρχιτεκτονική τους είναι ταυτόσημη (§4), τα δύο αυτά αρχεία εξετάζονται μαζί, με έμφαση στα σημεία όπου το ένα διαφέρει από το άλλο.

5.1 Δημιουργία σημείων Rhino: *lazy* με caching έναντι *eager* χωρίς caching

Η πιο ουσιώδης διαφορά εντοπίζεται στον κατασκευαστή της `MyPoint`. Στο **`v17_optimized.py`**, το πεδίο `self.id` αρχικοποιείται σε `None` και δημιουργείται ένα λεξικό `self._cache = {}`, το οποίο χρησιμοποιείται στη συνέχεια από τις ιδιότητες `boundary_hit`, `limits_bool_list`, `is_line` και `required_tangent_vector` ώστε ο υπολογισμός τους να γίνεται **μία μόνο φορά ανά σημείο** και να επιστρέφεται αποθηκευμένο αποτέλεσμα σε κάθε επόμενη κλήση:

```
self.id = None
self._cache = {}
...
@property
def boundary_hit(self):
    if 'bh' not in self._cache:
        tags = []
        ...
        self._cache['bh'] = "+".join(unique_tags)
    return self._cache['bh']
```

Listing 9: `v17_optimized.py`: lazy id + caching στην MyPoint

Στο **v17_copy.py**, αντίθετα, το αντικείμενο σημείου Rhino δημιουργείται **άμεσα** στον κατασκευαστή (`self.id = rs.AddPoint(rg_pt)`) και δεν υπάρχει κανένα `_cache`: κάθε ιδιότητα (π.χ. `boundary_hit`) υπολογίζεται εξ αρχής σε **κάθε** κλήση:

```
self.id = rs.AddPoint(rg_pt)
...
@property
def boundary_hit(self):
    tags = []
    if self.hit:
        ...
    # υπολογίζεται( από την αρχή σε κάθε πρόσβαση)
```

Listing 10: v17_copy.py: eager δημιουργία σημείου, χωρίς caching

Δεδομένου ότι ιδιότητες όπως `required_tangent_vector` και `boundary_hit` προσπελαύνονται επανειλημμένα – τόσο κατά την κατασκευή των τμημάτων (`MyCurve.rebuild`) όσο και σε *κάθε* από τους 5 κύκλους του βρόχου cross-fairing, για κάθε σημείο κάθε νομέα και κάθε ισάλου – η απουσία caching στο **v17_copy.py** συνεπάγεται πολλαπλάσιο πλήθος επαναλαμβανόμενων υπολογισμών ίδιων τιμών, καθώς και εκατοντάδες περιττά αντικείμενα σημείων Rhino (`rs.AddPoint`) για ενδιάμεσα/βοηθητικά `MyPoint` που ουδέποτε χρειάζονται να υπάρχουν ως ορατή γεωμετρία στο έγγραφο. Πρόκειται δηλαδή ακριβώς για το είδος βελτιστοποίησης που υποδηλώνει το προσωνύμιο «optimized» του πρώτου αρχείου.

Μία δεύτερη, συναφής διαφορά εντοπίζεται στη διαχείριση συντεταγμένων: το **v17_optimized.py** χρησιμοποιεί σταθερά πρόσβαση σε ιδιότητες του `rg.Point3d` (`coords.X`, `coords.Y`, `coords.Z`), ενώ το **v17_copy.py** εναλλάσσει αυτή την προσέγγιση με πρόσβαση μέσω `rs.PointCoordinates(...)` και δεικτοδότηση (`coords[0]`, `coords[1]`, `coords[2]`) σε αρκετά σημεία – δύο ισοδύναμοι αλλά μη ενοποιημένοι τρόποι χειρισμού συντεταγμένων, που στο **v17_optimized.py** έχουν ομογενοποιηθεί.

5.2 Πρόβλημα κωδικοποίησης χαρακτήρων στο `v17_copy.py`

Πέραν των αμιγώς λειτουργικών διαφορών, εντοπίστηκε ένα καθαρά τεχνικό ζήτημα: το αρχείο **v17_copy.py** είναι αποθηκευμένο με UTF-8 BOM και τα ελληνικά σχόλιά του εμφανίζουν *mojibake* (διπλή κωδικοποίηση UTF-8, δηλαδή κείμενο που κωδικοποιήθηκε σε UTF-8, στη συνέχεια διαβάστηκε εσφαλμένα ως Latin-1/Windows-1252 και ξανα-αποθηκεύτηκε σε UTF-8). Ενδεικτικά, η φράση *«αποφυγή crash»* εμφανίζεται στο αρχείο ως:

```
#€†…Î±ÏÎ¿ÏÏÎ³Î® crash, Î¸Î± †ƒ„ÎµÎ¾Î±Î½Î¹ÏÎµÎ¯ €Î±ÏÎ»Î¬ IndexError: list index out of range
```

Listing 11: Απόσπασμα σχολίου από το **v17_copy.py** όπως είναι αποθηκευμένο στο δίσκο

Το πρόβλημα περιορίζεται στα ελληνικά σχόλια (τεκμηρίωση) και δεν επηρεάζει τη λειτουργικότητα του κώδικα, καθώς η Python αγνοεί τα σχόλια κατά την εκτέλεση· είναι ωστόσο ενδεικτικό ότι το αρχείο πέρασε, σε κάποιο σημείο του κύκλου ανάπτυξής του, από κάποιο εργαλείο/αποθήκευση με εσφαλμένη ρύθμιση

κωδικοποίησης, κάτι που το **v17_optimized.py** (καθαρό UTF-8, χωρίς BOM) δεν παρουσιάζει.

5.3 Κοινή στρατηγική πρύμνης

Και τα δύο αρχεία αυτού του κεφαλαίου διαχειρίζονται τους ασυνεχείς νομείς της πρύμνης με τον ίδιο τρόπο: μετά την παραγωγή της επίπεδης επιφάνειας του καθρέφτη πρύμνης (`rs.AddPlanarSrf` πάνω στο profile, το transom και μία ευθεία σύνδεσης καταστρωμάτων, προβεβλημένα ακριβώς στο επίπεδο YZ μέσω `project_to_yz`), ακολουθεί ξεχωριστός βρόχος («STERN DISCONTINUOUS STATION SURFACES») που χτίζει, τμήμα προς τμήμα, επιφάνειες δικτύου (`rs.AddNetworkSrf`) ανάμεσα στον καθρέφτη πρύμνης και το πρώτο τμήμα κάθε ασυνεχούς νομέα που αγγίζει το profile, χρησιμοποιώντας ως V-καμπύλες τους ισάλους και τα τμήματα profile ανάμεσα στα δύο ορίων x:

```
u1_id = transom_copy
for station_data in discontinuous_stations_data:
    profile_intersecting_segments = [
        seg.id for seg in station_obj.segments
        if abs(seg.start_coords.Y) < 0.1 or abs(seg.end_coords.Y) < 0.1
    ]
    u2_id = ...  # ένωση των τμημάτων που αγγίζουν το profile
    v_curves = ίσαλοι[ + τμήματα profile ανάμεσα σε u1_x και station_x]
    network_srf = rs.AddNetworkSrf([u1_id, u2_id] + v_curves)
    u1_id = u2_id  # το τρέχον τμήμα γίνεται η νέα αφετηρία
```

Listing 12: Στρατηγική στεργανού δικτύου για την πρύμνη (ταυτόσημη σε optimized/copy)

6 `rhino_new_8.py`: γενικευμένη αρχιτεκτονική

Το **rhino_new_8.py** διατηρεί όλο τον κοινό πυρήνα του Κεφαλαίου 3 (συμπεριλαμβανομένου του lazy/caching μοντέλου `MyPoint` του **v17_optimized.py** – βλ. `self.id = None`, `self._cache = {}`, καθώς και μίας επιπλέον ιδιότητας `_compute_tangent` στη `MyPoint` που συγκεντρώνει σε ξεχωριστή μέθοδο τον υπολογισμό που στα άλλα δύο αρχεία γίνεται εξ ολοκλήρου εντός της `required_tangent_vector`), αλλά τον επεκτείνει σε δύο ανεξάρτητες κατευθύνσεις: την πύκνωση του πλέγματος νομέων και τη γενίκευση της παραγωγής επιφανειών ώστε να καλύπτει απευθείας ασυνεχείς νομείς.

6.1 Πύκνωση πλέγματος νομέων: `create_intermediate_stations`

Αμέσως μετά το τέλος του cross-fairing βρόχου, το **rhino_new_8.py** εισάγει μία συνάρτηση που δεν υπάρχει καθόλου στα άλλα δύο αρχεία: `create_intermediate_stations(existing_statio`
`waterline_curves, fos_curve, fob_curve, profile_curve, ship_limits_matrix, n_intermediate=3)`.
Σκοπός της είναι να παρεμβάλλει, ανάμεσα σε κάθε ζεύγος διαδοχικών, ήδη υπαρχόντων νομέων, τρεις νέους ενδιάμεσους νομείς ισοκατανεμημένους κατά x:

```
def create_intermediate_stations(existing_stations, waterline_curves, fos_curve,
                                 fob_curve, profile_curve, ship_limits_matrix,
```

```
                                  n_intermediate=3):
    """
    Για κάθε ζεύγος γειτονικών υπαρχόντων νομέων ταξινομημένων( κατά x),
    εισάγει n_intermediate νέους, ισοκατανεμημένους νομείς.
    1. Τομή FOB/profile/FOS με το νέο κατακόρυφο επίπεδο x -> όρια (intercepts).
    2. Τομή των τρεχόντων εξομαλυμένων ισάλων με το ίδιο επίπεδο -> εσωτερικά σημεία.
    3. Ίδια λογική σύζευξηςκατασκευής/ MyCurve όπως στο build_station_segments,
       ώστε οι νέοι νομείς να δουλεύουν απρόσκοπτα με το fairing/lofting.
    """
```

Listing 13: create_intermediate_stations: αρχή της συνάρτησης

Κάθε νέος νομέας κατασκευάζεται με τον ίδιο τρόπο που θα κατασκευαζόταν ένας «κανονικός» νομέας από τα αρχικά δεδομένα (§3.3): τα όρια της (πλέον εξομαλυμένης) γεωμετρίας δίνουν τα σημεία-τομής με FOB/profile/FOS, ενώ οι τρέχοντες, ήδη κλειδωμένοι ίσαλοι δίνουν τα εσωτερικά σημεία της καμπύλης. Οι νέοι νομείς αριθμούνται από 10000 και πάνω, ώστε να μην συγκρούονται με την αρίθμηση των αρχικών νομέων. Αποτέλεσμα είναι ένα πυκνότερο πλέγμα νομέων, το οποίο μειώνει το μήκος (και άρα τη γεωμετρική «ελευθερία») κάθε μεμονωμένου επιφανειακού τμήματος `MySurface` στην κύρια ενότητα παραγωγής επιφανειών, βελτιώνοντας δυνητικά την πιστότητα της τελικής επιφάνειας σε περιοχές έντονης καμπυλότητας.

6.2 Γενίκευση της `MySurface` για απευθείας χειρισμό ασυνεχών νομέων

Στα **`v17_optimized.py`/`v17_copy.py`**, ένα ζεύγος νομέων όπου έστω ο ένας είναι ασυνεχής προκαλεί άμεση παράλειψη (`"SKIPPED: Discontinuous"`) στη γενική ροή παραγωγής επιφανειών, με την πρύμνη να αντιμετωπίζεται εξ ολοκλήρου ξεχωριστά, στο τέλος του script (§5). Το **`rhino_new_8.py`** καταργεί αυτόν τον περιορισμό, προσθέτοντας στην κλάση `MySurface` πέντε νέες μεθόδους:

- `_collect_segs(station_data)` *(static)* – συλλέγει και ενώνει όλα τα τμήματα Rhino ενός νομέα σε μία ενιαία καμπύλη, με σαφή διαβαθμισμένη στρατηγική αστοχίας: αν η πλήρης ένωση αποτύχει, χρησιμοποιεί τη μεγαλύτερη επιμέρους αλυσίδα.
- `_collect_segs_by_part(station_data)` *(static)* – η αντίστοιχη εκδοχή για **ασυνεχείς** νομείς: αντί να επιχειρεί ενιαία ένωση, ομαδοποιεί τα τμήματα ανά `Part_N` (εξάγοντας τον αριθμό τμήματος από το όνομα της καμπύλης) και επιστρέφει λεξικό `{part_idx: coerced_curve}`.
- `_trim_curve_by_z(curve, z_lo, z_hi, tol=0.1)` *(static)* – κόβει μία καμπύλη στο τμήμα ανάμεσα σε δύο οριζόντια επίπεδα, μέσω τομής με τα επίπεδα αυτά και `Curve.Trim` στην αντίστοιχη παραμετρική περιοχή.
- `_loft_pair(c1, c2, n1, n2, suffix="")` – ενοποιεί σε μία μέθοδο την επαναλαμβανόμενη ακολουθία «`CreateFromLoft` → `AddBrep` → ονομασία → κατοπτρισμός», η

οποία στα άλλα δύο αρχεία επαναλαμβανόταν αυτούσια σε πολλά σημεία του κώδικα (loft πλώρης, loft πρύμνης, δίκτυα πρύμνης).

- `_close_gaps(...)` – κλείνει τυχόν κενά που προκύπτουν όταν οι δύο πλευρές ενός ζεύγους νομέων δεν καλύπτουν το ίδιο ακριβώς εύρος z (π.χ. όταν ο ένας νομέας έχει περισσότερα «κομμάτια» από τον άλλο).

Η μέθοδος `create_surface()` του **`rhino_new_8.py`** χρησιμοποιεί πλέον αυτά τα εργαλεία ώστε, για ασυνεχείς νομείς, να ελέγχει και να επεκτείνει (`_check_waterproof` / `_extend_station`) **κάθε** επιμέρους τμήμα ξεχωριστά – όχι μόνο το «αντιπροσωπευτικό» τμήμα, όπως στα άλλα δύο αρχεία:

```
for sd in [self.x1_data, self.x2_data]:
    if sd is None or not sd['discontinuous']:
        continue
    rep = sd['curve']
    for mc in sd.get('all_curves', []):
        if mc is rep:
            continue  # ήδη έγινε παραπάνω
        wp = self._check_waterproof(mc)
        if not all(wp):
            self._extend_station(mc)
```

Listing 14: `rhino_new_8.py`: επέκταση κάθε τμήματος ασυνεχούς νομέα, όχι μόνο του αντιπροσωπευτικού

Για να είναι εφικτή αυτή η γενίκευση, το λεξικό `processed_stations` στο **`rhino_new_8.py`** αποκτά ένα επιπλέον κλειδί, `'all_curves'`, που κρατά ολόκληρη την ομάδα τμημάτων ενός αριθμού νομέα (όχι μόνο το αντιπροσωπευτικό), ώστε οι νέες στατικές μέθοδοι της `MySurface` να έχουν πρόσβαση σε αυτά:

```
processed_stations.append({
    'number': num,
    'curve': representative,
    'discontinuous': is_discontinuous,
    'all_curves': group,       # πλήρης λίστα, για lofting μέσω _collect_segs
})
```

Listing 15: Επιπλέον κλειδί 'all_curves' στο processed_stations (μόνο στο rhino_new_8.py)

6.3 Ενοποιημένη στρατηγική πρύμνης: *blended loft* αντί για δίκτυο επιφανειών

Η τελική ενότητα του **`rhino_new_8.py`** («*TRANSOM PLANAR SURFACE + STERN LOFT TO FIRST STATION*») αντικαθιστά πλήρως τη λογική «δικτύου επιφανειών ανά τμήμα» των δύο άλλων αρχείων. Αντί να χτίζει ξεχωριστή επιφάνεια δικτύου για κάθε τμήμα ασυνεχούς νομέα ανάμεσα στον καθρέφτη πρύμνης και τον πρώτο νομέα, το **`rhino_new_8.py`**:

1. Παράγει, όπως και τα άλλα δύο αρχεία, την επίπεδη επιφάνεια του καθρέφτη πρύμνης μέσω `rs.AddPlanarSrf`.

2. Συλλέγει την (ενοποιημένη πλέον, μέσω `MySurface._collect_segs`) καμπύλη του πρώτου νομέα.
3. Κατασκευάζει μία **ενδιάμεση** τομή, παρεμβάλλοντας γραμμικά (κατά μέσο όρο Y, Z σε 31 σημεία δειγματοληψίας) ανάμεσα στην καμπύλη transom και την καμπύλη του πρώτου νομέα, στο μέσο x των δύο.
4. Εκτελεί ένα ενιαίο `rg.Brep.CreateFromLoft` πάνω και στις τρεις τομές (transom, ενδιάμεση, πρώτος νομέας), παράγοντας μία συνεχή επιφάνεια πρύμνης με μία μόνο κλήση, αντί για αλυσίδα επιμέρους δικτύων.

```
_scurve, _sinfo = MySurface._collect_segs(_sdata)   # πρώτος νομέας, ενοποιημένος
_stern_secs = [_transom_rh]
for k in range(31):
    t = k / 30.0
    p1 = _transom_rh.PointAt(...)   # σημείο πάνω στο transom
    p2 = _scurve.PointAt(...)       # αντίστοιχο σημείο πάνω στον πρώτο νομέα
    mid_x = (transom_x + station_x) / 2.0
    mid_pts.append(rg.Point3d(mid_x, (p1.Y+p2.Y)/2.0, (p1.Z+p2.Z)/2.0))
mid_curve = rg.Curve.CreateInterpolatedCurve(mid_pts, 3)
_stern_secs.append(mid_curve)
_stern_secs.append(_scurve)

loft = rg.Brep.CreateFromLoft(_stern_secs, rg.Point3d.Unset, rg.Point3d.Unset,
                              rg.LoftType.Normal, False)
```

Listing 16: `rhino_new_8.py`: ενιαίος blended loft πρύμνης (απόσπασμα)

Πρόκειται για μία εννοιολογικά διαφορετική προσέγγιση του ίδιου προβλήματος: αντί να «χτίζεται» η πρύμνη τμήμα-τμήμα ακολουθώντας τα φυσικά όρια των ασυνεχών κομματιών κάθε νομέα (προσέγγιση **`v17_optimized`/`v17_copy`**, πιστή στη διαμερισμένη γεωμετρία αλλά με περισσότερες, μικρότερες επιφάνειες προς ραφή), το **`rhino_new_8.py`** «γεφυρώνει» ολόκληρη την περιοχή πρύμνης με μία ενιαία, ομαλή επιφάνεια lofting μέσω μίας τεχνητής ενδιάμεσης τομής – λιγότερες, μεγαλύτερες επιφάνειες, με το ρίσκο να «στρογγυλοποιεί» τοπικές λεπτομέρειες της πρυμναίας γεωμετρίας που το δίκτυο τμημάτων θα αναπαρήγαγε πιστότερα.

7 Συγκεντρωτική σύγκριση

Χαρακτηριστικό	v17_optimized	v17_copy	rhino_new_8
Γραμμές κώδικα	2712	2655	3035
Κωδικοποίηση αρχείου	UTF-8, χωρίς BOM	UTF-8 BOM, mojibake στα ελληνικά σχόλια	UTF-8, χωρίς BOM
Δημιουργία `MyPoint.id`	Lazy (`None`, δημιουργείται όταν χρειαστεί)	Eager (`rs.AddPoint` στον κατασκευαστή)	Lazy, όπως optimized

Χαρακτηριστικό	v17_optimized	v17_copy	rhino_new_8
Caching ιδιοτήτων (`_cache`)	Ναι	Όχι	Ναι
Πρόσβαση συντεταγμένων	Ομοιόμορφα μέσω `.X/.Y/.Z`	Μεικτή (`.X/.Y/.Z` και `rs.PointCoordinates`+δεικτοδότηση)	Όπως optimized
`clamp_waterlines_y`	Ναι	Όχι	Όχι (αντ' αυτού άλλη λογική στο `_collect_segs`)
`create_intermediate_stations`	Όχι	Όχι	Ναι (πύκνωση πλέγματος νομέων)
`MySurface`: ασυνεχείς νομείς	Παραλείπονται στη γενική ροή	Παραλείπονται στη γενική ροή	Ενσωματωμένοι απευθείας
Πρόσθετες στατικές μέθοδοι `MySurface`	–	–	`_collect_segs`, `_collect_segs_by_part`, `_trim_curve_by_z`, `_loft_pair`, `_close_gaps`
Επιφάνεια πρύμνης	Δίκτυο επιφανειών ανά τμήμα ασυνεχούς νομέα	Ίδια με optimized	Ενιαίο blended loft (transom → ενδιάμεση τομή → πρώτος νομέας)

8 Συμπεράσματα

Και τα τρία αρχεία μοιράζονται τον ίδιο σχεδιαστικό πυρήνα: ένα αντικειμενοστραφές μοντέλο τριών επιπέδων (`MyPoint` → `MySegment` → `MyCurve`) στο οποίο κάθε σημείο «γνωρίζει» τους γεωμετρικούς περιορισμούς και την απαιτούμενη εφαπτομενική συνθήκη του, μία ενιαία συνάρτηση εξομάλυνσης εφαρμόσιμη οπουδήποτε, και έναν επαναληπτικό βρόχο αμοιβαίας εξομάλυνσης νομέων/ισάλων (cross-fairing) που κλειδώνει σε ένα τελικό, μη-εξομαλυμένο πέρασμα παρεμβολής. Η σχεδιαστική αυτή επιλογή – να κωδικοποιείται η γεωμετρική «νοημοσύνη» στο ίδιο το σημείο, αντί σε ξεχωριστή, ανά περίπτωση λογική per-καμπύλη – είναι αυτή που επιτρέπει την επαναχρησιμοποίηση της `universal_fair_function` και του `MySurface` σε ολόκληρο το εύρος γεωμετριών της γάστρας, από την ίσια καρίνα μέχρι τη σύνθετη, πολυτμηματική πρύμνη.

Οι δύο εκδοχές «v17» διαφέρουν αποκλειστικά σε επίπεδο υλοποίησης απόδοσης (lazy δημιουργία αντικειμένων Rhino με caching έναντι άμεσης δημιουργίας χωρίς caching), με το **`v17_copy.py`** να φέρει επιπλέον ένα ανεξάρτητο, καθαρά τεχνικό ζήτημα κωδικοποίησης χαρακτήρων στα σχόλιά του. Το **`rhino_new_8.py`** αντιπροσωπεύει τον πιο εκτεταμένο κλάδο, με δύο ουσιώδεις προσθήκες αρχιτεκτονικής – προαιρετική πύκνωση πλέγματος νομέων και γενικευμένη, ενιαία διαχείριση ασυνεχών νομέων μέσα στην ίδια την `MySurface`, με αντίστοιχη απλοποίηση/ενοποί-

ηση της τελικής επιφάνειας πρύμνης σε ένα ενιαίο loft.